

Day 6

Synchronous Single port RAM

Verilog Code:

```
module sync_ram #(parameter DEPTH = 16,  
parameter WIDTH = 8) ( input clk, we,  
input [$clog2(DEPTH)-1:0] addr_bus,  
input [WIDTH-1:0] data_in,  
output reg [WIDTH-1:0] data_out);
```

```
reg [WIDTH-1:0] mem [0:DEPTH-1];
```

```
always @(posedge clk) begin  
    if(we)  
        mem[addr_bus] <= data_in;  
    else  
        data_out <= mem[addr_bus];  
    end  
endmodule
```

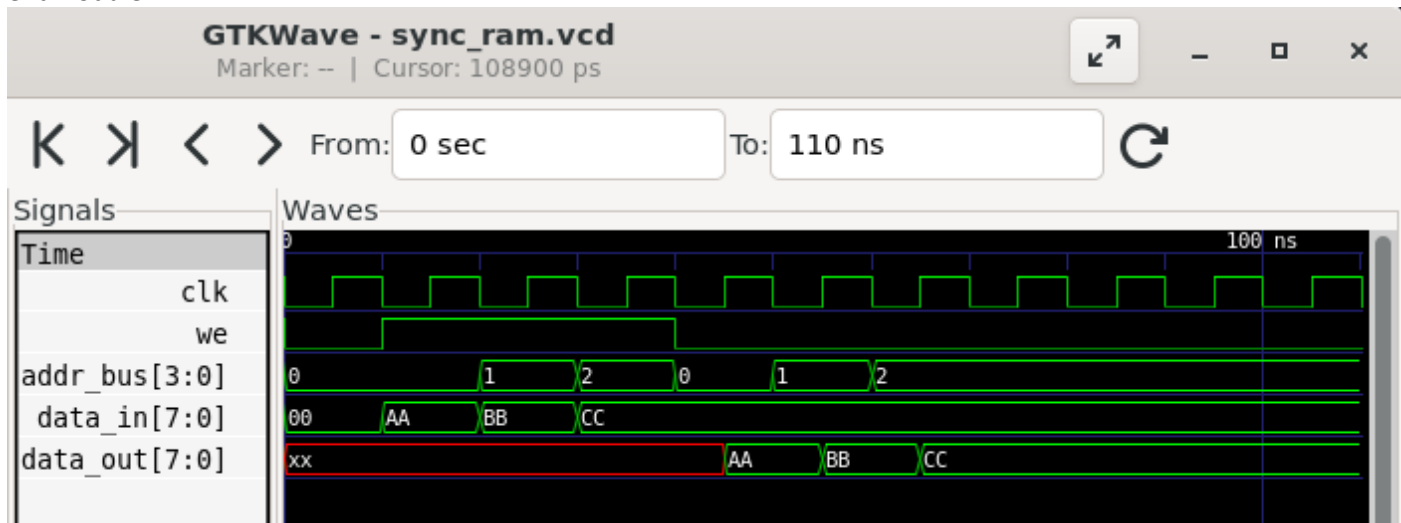
Testbench:

```
`timescale 1ns/1ps  
module tb;  
parameter WIDTH = 8;  
parameter DEPTH = 16;  
reg clk, we;  
reg [$clog2(DEPTH)-1:0] addr_bus;  
reg [WIDTH-1:0] data_in;  
wire [WIDTH-1:0] data_out;  
  
sync_ram uut(.clk(clk), .we(we), .addr_bus(addr_bus), .data_in(data_in), .data_out(data_out));  
initial begin  
    clk = 0;  
    forever #5 clk = ~clk;  
end  
initial begin  
    $dumpfile("sync_ram.vcd");  
    $dumpvars(0, tb);  
end  
initial begin  
    we=0; addr_bus=0; data_in=0;  
    #10;  
    we=1; addr_bus=4'h0; data_in=8'hAA;  
    #10;  
    we=1; addr_bus=4'h1; data_in=8'hBB;  
    #10;  
    we=1; addr_bus=4'h2; data_in=8'hCC;  
    #10;  
    we=0;  
    addr_bus=4'h0;  
    #10;  
    $display("Address 0 contains: %h", data_out);  
    addr_bus=4'h1;  
    #10;  
    $display("Address 1 contains: %h", data_out);  
    addr_bus=4'h2;  
    #10;
```

```

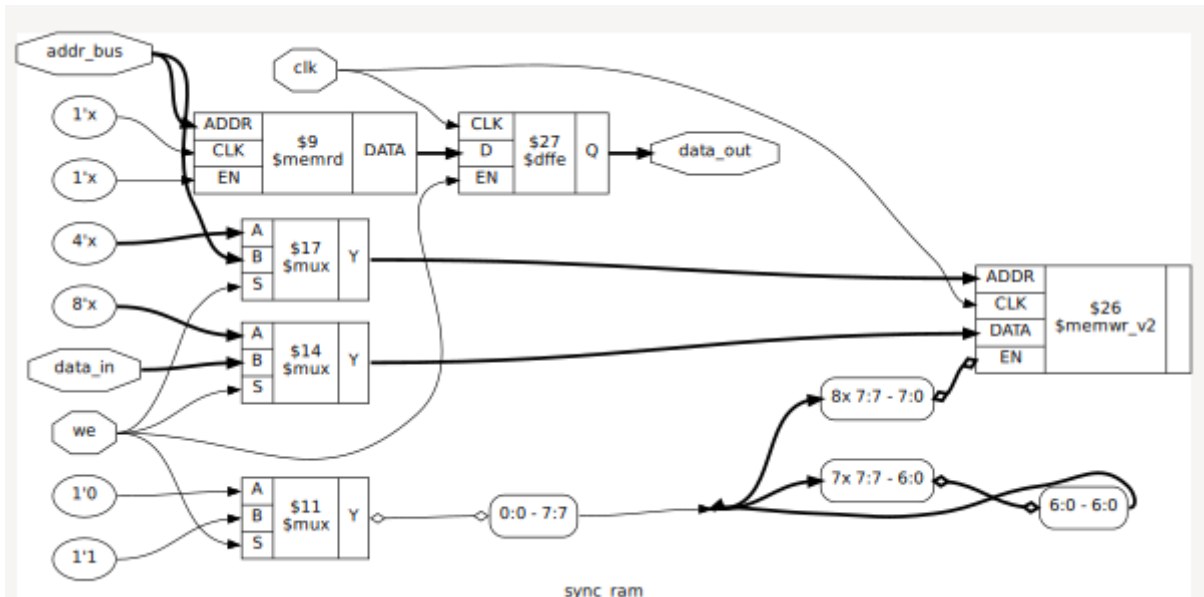
$display("Address 2 contains: %h", data_out);
#40 $finish;
end
endmodule

```



Simulation and Synthesis output:

VCD info: dumpfile sync_ram.vcd opened for output.
Address 0 contains: aa
Address 1 contains: bb
Address 2 contains: cc



2.26. Printing statistics.

=== sync_ram ===

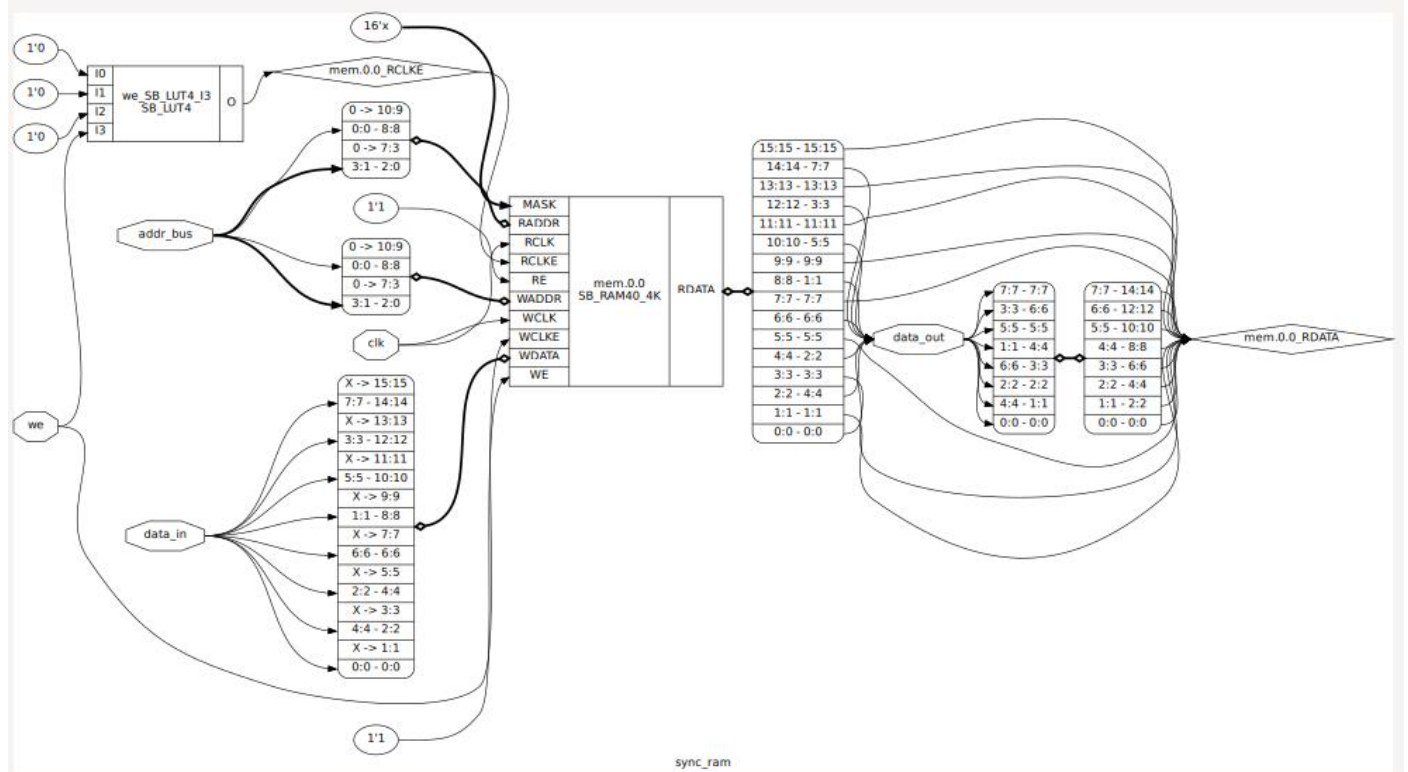
```

Number of wires:      24
Number of wire bits:  130
Number of public wires: 5
Number of public wire bits: 22
Number of memories:   1
Number of memory bits: 128
Number of processes:   0
Number of cells:      10
  $dff                 4
  $memrd                1

```

\$memwr_v2 1
\$mux 4

Command: yosys -p "synth_ice40 -top sync_ram; show" sync_ram.v



2.48. Printing statistics.

=== sync_ram ===

Number of wires:	7
Number of wire bits:	39
Number of public wires:	7
Number of public wire bits:	39
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	2
SB_LUT4	1
SB_RAM40_4K	1

FIFO

```

module fifo #(
  parameter DEPTH = 8,
  parameter WIDTH = 8)(
  input clk, rst,
  input wr_en, rd_en,
  input [WIDTH-1:0] din,
  output reg [WIDTH-1:0] dout,
  output full, empty);

  reg [WIDTH-1:0] mem [0:DEPTH-1];
  reg [$clog2(DEPTH):0] wr_ptr, rd_ptr, count;

  // write
  always @(posedge clk) begin

```

```

if(rst) wr_ptr <= 0;
else if(wr_en && !full) begin
    mem[wr_ptr] <= din;
    wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
end
end
//read
always @(posedge clk) begin
    if(rst) rd_ptr <= 0;
    else if(rd_en && !empty) begin
        dout <= mem[rd_ptr];
        rd_ptr <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;
    end
end
always @(posedge clk) begin
    if(rst) count <= 0;
    else begin
        case({wr_en & !full, rd_en & !empty})
            2'b10: count <= count+1;
            2'b01: count <= count-1;
            2'b11: count <= count;
            default: count <= count;
        endcase
    end
end

assign full = (count == DEPTH);
assign empty = (count == 0);
endmodule

```

Testbench:

```
`timescale 1ns / 1ps
```

```

module tb;
    // 1. Parameters & Signals
    parameter DEPTH = 8;
    parameter WIDTH = 8;

    reg clk, rst;
    reg wr_en, rd_en;
    reg [WIDTH-1:0] din;
    wire [WIDTH-1:0] dout;
    wire full, empty;
    wire [$clog2(DEPTH):0] count;

    // 2. Instantiate the FIFO
    fifo #(DEPTH, WIDTH) uut (
        .clk(clk), .rst(rst),
        .wr_en(wr_en), .rd_en(rd_en),
        .din(din), .dout(dout),
        .full(full), .empty(empty)
    );

    // 3. Clock Generation (10ns period)
    always #5 clk = ~clk;
    initial begin
        $dumpfile("fifo.vcd");
    end
endmodule

```

```

$dumpvars(0, tb);
end
// 4. Test Stimulus
initial begin
    // Initialize
    clk = 0; rst = 1; wr_en = 0; rd_en = 0; din = 0;
    #20 rst = 0; // Release reset

    // --- TEST 1: Fill the FIFO ---
    $display("--- Starting Fill Test ---");
    repeat (DEPTH) begin
        @(negedge clk);
        if (!full) begin
            wr_en = 1;
            din = $random % 256;
            $display("Writing: %h | Count expected to increase", din);
        end
    end

    @(negedge clk);
    wr_en = 0;
    #5;
    if (full) $display("FIFO is now FULL as expected.");

    // --- TEST 2: Overflow Attempt ---
    @(negedge clk);
    wr_en = 1; din = 8'hFF;
    #10;
    wr_en = 0;
    $display("Attempted write to full FIFO. Full flag is: %b", full);

    // --- TEST 3: Drain the FIFO ---
    $display("--- Starting Drain Test ---");
    repeat (DEPTH) begin
        @(negedge clk);
        if (!empty) begin
            rd_en = 1;
        end
        @(posedge clk); #1; // Wait for data to clock out
        $display("Read: %h", dout);
    end

    @(negedge clk);
    rd_en = 0;
    #5;
    if (empty) $display("FIFO is now EMPTY as expected.");

    // --- TEST 4: Simultaneous Read and Write (Streaming) ---
    $display("--- Starting Simultaneous Read/Write Test ---");

    // First, put at least one item in so it's not empty
    @(negedge clk);
    wr_en = 1; rd_en = 0; din = 8'h11;

    @(negedge clk);
    // Now turn BOTH on

```

```

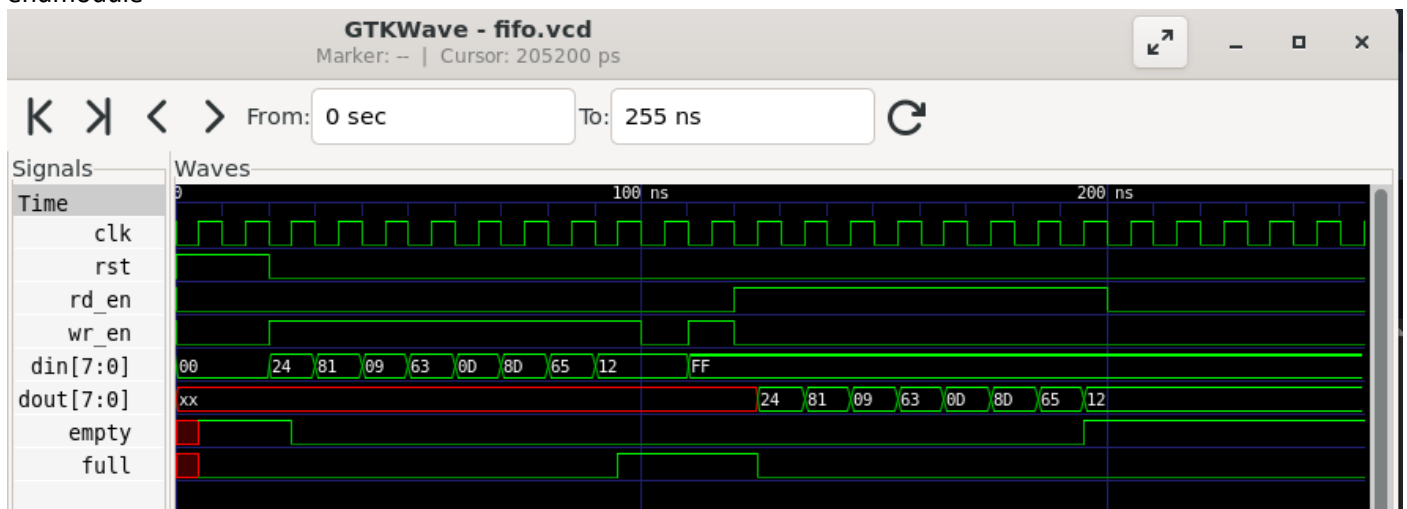
wr_en = 1;
rd_en = 1;
din = 8'h22; // Writing 22 while reading 11

repeat (3) begin
    @(negedge clk);
    din = din + 1; // Change data every cycle
    $display("Streaming - Count: %d | Full: %b | Empty: %b", count, full, empty);
end

@(negedge clk);
wr_en = 0;
rd_en = 0;

#50 $finish;
end
endmodule

```



VCD info: dumpfile fifo.vcd opened for output.

--- Starting Fill Test ---

Writing: 24 | Count expected to increase

Writing: 81 | Count expected to increase

Writing: 09 | Count expected to increase

Writing: 63 | Count expected to increase

Writing: 0d | Count expected to increase

Writing: 8d | Count expected to increase

Writing: 65 | Count expected to increase

Writing: 12 | Count expected to increase

FIFO is now FULL as expected.

Attempted write to full FIFO. Full flag is: 1

--- Starting Drain Test ---

Read: 24

Read: 81

Read: 09

Read: 63

Read: 0d

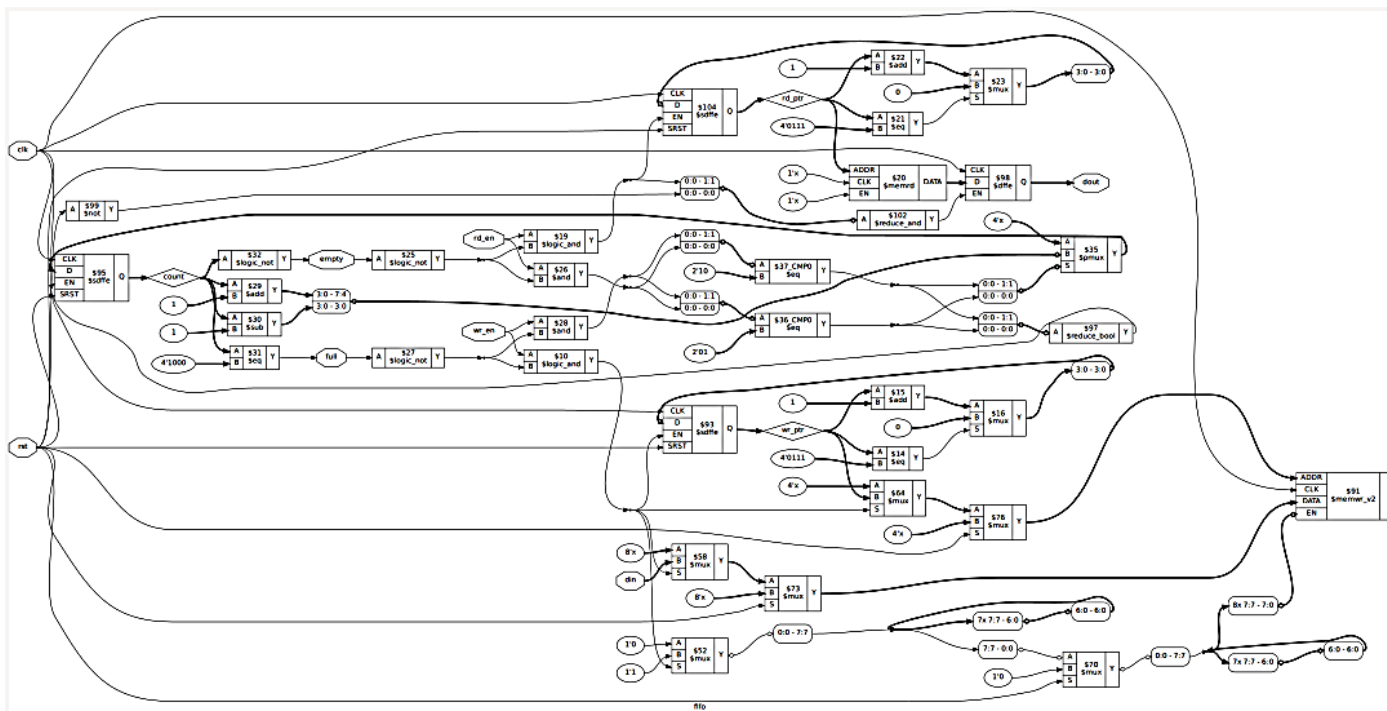
Read: 8d

Read: 65

Read: 12

FIFO is now EMPTY as expected.

Command: yosys -p "hierarchy -top fifo; proc; opt; show" fifo.v



=== fifo ===

Number of wires:	81
Number of wire bits:	464
Number of public wires:	11
Number of public wire bits:	34
Number of memories:	1
Number of memory bits:	64
Number of processes:	0
Number of cells:	46
\$add	3
\$and	2
\$dff	7
\$eq	5
\$logic_and	2
\$logic_not	5
\$memrd	1
\$memwr_v2	1
\$mux	18
\$pmux	1
\$sub	1

Key Learnings from today:

Learning 1 — 2D reg array infers a memory block, not individual FFs

In the RAM:

Number of memories:	1
Number of memory bits:	128
\$memrd	1
\$memwr_v2	1

Yosys recognised reg [7:0] mem [0:15] as a memory and kept it as a single \$memrd/\$memwr_v2 block — not 128 individual flip-flops. This is memory inference. The synthesiser is smart enough to know this should map to a RAM block, not a sea of FFs.

Same thing happened in the FIFO:

Number of memories:	1
Number of memory bits:	64 ← 8 locations × 8 bits

Learning 2 — Technology mapping collapses everything

I ran `synth_ice40` on the RAM and got:

```
SB_LUT4      1
SB_RAM40_4K  1
```

From 10 generic cells down to **2 technology-specific cells**. The entire RAM collapsed into one `SB_RAM40_4K` — a hardened block RAM primitive in the iCE40 FPGA fabric. This is what happens in real FPGA synthesis — the behavioral RTL maps to dedicated silicon blocks that are already manufactured on the chip, not built from LUTs.

This is the difference between:

- `synth` — generic gates, technology-independent
- `synth_ice40` / `synth_xilinx` — maps to real FPGA primitives

Learning 3 — FIFO cells tell the complete story

Cell	Count	What it represents
<code>\$dff</code>	7	<code>wr_ptr(4b) + rd_ptr(4b) + count(4b)</code> — but shared bits = 7 FFs after opt
<code>\$add</code>	3	<code>ptr+1</code> for <code>wr</code> , <code>ptr+1</code> for <code>rd</code> , <code>count+1</code>
<code>\$sub</code>	1	<code>count-1</code> for read
<code>\$eq</code>	5	comparators for full, empty, and case conditions
<code>\$mux</code>	18	all the if/else and case mux logic
<code>\$pmux</code>	1	the case statement with 4 conditions (parallel mux)
<code>\$memrd</code>	1	the memory read port
<code>\$memwr_v2</code>	1	the memory write port
<code>\$logic_not</code>	5	all the !full and !empty inversions

Every line of the RTL is accounted for in this list. Nothing hidden.

Learning 4 — `$pmux` vs `$mux`

We saw both in the FIFO. The difference:

- `$mux` — 2-input mux, one select line — from simple if/else
- `$pmux` — parallel mux, multiple select lines — from case with multiple branches

The count logic `case({wr_en & !full, rd_en & !empty})` with 4 branches became exactly one `$pmux`.

Learning 5 — The circular pointer trick

I improved on the basic design by adding:

```
wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
```

This makes the pointer wrap around instead of overflowing. In the synthesis report this shows up as an extra `$eq` comparator plus a `$mux` — one to detect the wrap condition, one to select between 0 and `ptr+1`. A small but real hardware cost for correct circular behaviour.

Learning 6 — Self-checking testbench with `$display`

I used `$display` to print read/write values and flag states directly in the terminal. This is the first step toward a proper self-checking testbench — instead of manually reading the waveform, the simulation tells you if it passed. The output showed:

```
FIFO is now FULL as expected.
```

```
Attempted write to full FIFO. Full flag is: 1
```

```
FIFO is now EMPTY as expected.
```

All three corner cases passed — full protection, overflow protection, and empty protection all working correctly.